Caso de Estudio 2

Allison Romero Jimenez

Emilio Conejo Fernandez

Isaac Vargas Cobar

Universidad CENFOTEC

Estructuras de Datos 2

David Campos Castro

Fecha: Marzo, 2026

Tabla de Contenido

Capítulo 1 4
Introducción 4
Capítulo 2 5
Desarrollo 5
Capítulo 3 6
Requerimientos Funcionales 6
Capítulo 4 7
Requerimientos No Funcionales 7
Requerimientos de Sistema 8
Capítulo 5 8
Limitaciones y Restricciones 8
Capítulo 6 9
UML (Diagrama de Clases) 9
Capítulo 7 11
Casos de Prueba 11
Capítulo 8 16
Recomendaciones 16
Lista de Referencias 17
Git Hub
Enlace al repositorio de Git Hub 17

Lista de figuras

Figura 0 ..... **9**
Figura 1 ..... **10**
Figura 2 ..... **12**
Figura 3 ..... **13**
Figura 4 ..... **14**
Figura 5 ..... **14**

Capítulo 1

Introducción

La presente práctica tiene como objetivo desarrollar un programa que simule el funcionamiento básico de un cajero automático mediante la implementación de un algoritmo voraz. Este tipo de algoritmo se caracteriza por seleccionar en cada paso la opción que parece ser la mejor solución inmediata, con el fin de construir una solución global al problema planteado (Cormen et al., 2022).

En el contexto de los cajeros automáticos, el algoritmo voraz se utiliza para determinar la combinación de billetes que se deben dispensar cuando un cliente solicita retirar una determinada cantidad de dinero. El algoritmo prioriza siempre el uso de los billetes de mayor denominación disponibles, con el objetivo de satisfacer la solicitud del cliente utilizando la menor cantidad posible de billetes.

Para el desarrollo de esta práctica se implementó una aplicación de consola en el lenguaje de programación Java utilizando el JDK 21. El sistema permite administrar el inventario de billetes del cajero automático, visualizar la cantidad disponible de cada denominación y realizar retiros de dinero aplicando el algoritmo voraz.

Además, se incorporaron mecanismos de validación de datos con el fin de evitar errores durante la ejecución del programa, garantizando que el sistema sea capaz de manejar entradas incorrectas sin provocar fallos o interrupciones inesperadas.

Capítulo 2

Desarrollo

El sistema desarrollado simula el comportamiento básico de un cajero automático que permite a los usuarios realizar operaciones relacionadas con el manejo de billetes y el retiro de dinero.

El programa se ejecuta mediante una interfaz de consola que presenta al usuario un menú principal con diferentes opciones. A través de este menú es posible acceder a las funcionalidades principales del sistema, las cuales incluyen la gestión del inventario de billetes del cajero automático y la posibilidad de retirar dinero.

El sistema implementa el principio de algoritmo voraz para determinar la cantidad de billetes que deben entregarse al cliente. Este algoritmo funciona seleccionando primero la mayor denominación de billete disponible que no supere el monto restante solicitado (Goodrich et al., 2014). Posteriormente, el proceso se repite con las siguientes denominaciones hasta completar la cantidad solicitada o determinar que no es posible dispensar el monto requerido.

Las denominaciones utilizadas en el sistema corresponden a los billetes comúnmente disponibles en cajeros automáticos en Costa Rica: ₡20000, ₡10000, ₡5000, ₡2000 y ₡1000.

Para organizar el código del sistema se utilizaron diferentes clases que separan las responsabilidades del programa. La clase principal se encarga de iniciar la ejecución del sistema, la clase encargada del menú gestiona la interacción con el usuario y la clase del cajero contiene la lógica del algoritmo voraz y la administración del inventario de billetes (Cormen et al., 2022).

Adicionalmente, se implementaron diversas validaciones para asegurar que el sistema funcione de manera correcta incluso cuando el usuario ingresa datos incorrectos, como

letras en lugar de números, montos negativos o cantidades que no pueden ser dispensadas por el cajero automático.

Capítulo 3

Requerimientos Funcionales

Los requerimientos funcionales describen las funciones que el sistema debe cumplir para satisfacer las necesidades planteadas en la práctica.

RF1. El sistema debe mostrar un menú principal que permita al usuario seleccionar entre las diferentes opciones disponibles.

RF2. El sistema debe permitir acceder a un submenú de mantenimiento.

RF3. El sistema debe permitir agregar billetes al inventario del cajero automático indicando la cantidad de cada denominación.

RF4. El sistema debe permitir visualizar la cantidad de billetes disponibles en el cajero automático.

RF5. El sistema debe permitir al usuario solicitar el retiro de una cantidad determinada de dinero.

RF6. El sistema debe calcular la cantidad de billetes a entregar utilizando el algoritmo voraz.

RF7. El sistema debe mostrar en pantalla la cantidad de billetes de cada denominación que serán entregados al usuario.

RF8. El sistema debe actualizar el inventario de billetes después de cada retiro realizado.

RF9. El sistema debe indicar cuando no es posible dispensar la cantidad solicitada por el usuario.

RF10. El sistema debe permitir salir del programa cuando el usuario lo desee.

Capítulo 4

Requerimientos No Funcionales

Los requerimientos no funcionales describen las características de calidad del sistema.

RNF1. El sistema debe desarrollarse utilizando el lenguaje de programación Java.

RNF2. El sistema debe ejecutarse utilizando el entorno Java Development Kit versión 21.

RNF3. El programa debe ejecutarse en modo consola.

RNF4. El sistema debe manejar adecuadamente errores de entrada de datos por parte del usuario.

RNF5. El sistema debe evitar que el programa se detenga inesperadamente ante datos incorrectos.

RNF6. El código fuente debe organizarse en clases que separen las responsabilidades del sistema.

RNF7. El código debe almacenarse en un repositorio utilizando Git y GitHub para control de versiones.

Requerimientos de Sistema

Para el correcto funcionamiento del sistema del cajero automático se requiere el siguiente entorno de desarrollo y ejecución:

- **Lenguaje de programación:** Java
- **Versión del JDK:** JDK 21 o superior
- **Entorno de desarrollo (IDE):** IntelliJ IDEA o cualquier IDE compatible con Java
- **Sistema de control de versiones:** Git
- **Repositorio del proyecto:** GitHub
- **Sistema operativo compatible:** Windows, Linux o macOS
- **Memoria mínima recomendada:** 4 GB de RAM
- **Procesador:** Dual Core o superior(Sugerido).

Capítulo 5

Limitaciones y Restricciones

El sistema desarrollado presenta ciertas limitaciones que deben ser consideradas durante su uso y desarrollo.

Una de las principales limitaciones del sistema es que se trata de una simulación simplificada de un cajero automático y no contempla aspectos relacionados con seguridad bancaria, autenticación de usuarios o conexión con bases de datos.

Otra limitación es que el sistema trabaja únicamente con cinco denominaciones de billetes específicas: ₡20000, ₡10000, ₡5000, ₡2000 y ₡1000.

Además, el algoritmo voraz utilizado en el sistema selecciona siempre la mayor denominación posible en cada paso. Aunque este enfoque permite obtener una solución

rápida y sencilla, no necesariamente garantiza una distribución equilibrada de los billetes disponibles en el cajero automático.

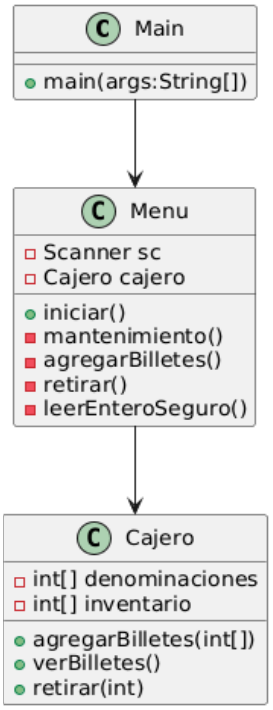
Finalmente, el sistema se ejecuta exclusivamente en consola y no cuenta con una interfaz gráfica de usuario.

Capítulo 6

UML (Diagrama de Clases)

Figura 0

Representación del Diagrama UML



Nota: Se asignaron las diferentes clases utilizadas

El diagrama UML de clases representa la estructura del sistema desarrollado para simular el funcionamiento de un cajero automático utilizando un algoritmo voraz.

El sistema está compuesto por tres clases principales: **Main**, **Menu** y **Cajero**. La clase **Main** es el punto de inicio del programa y se encarga de ejecutar el sistema. La clase **Menu** gestiona la interacción con el usuario mediante el menú principal y el submenú de mantenimiento. Por su parte, la clase **Cajero** contiene la lógica principal del sistema, incluyendo la administración del inventario de billetes y la implementación del algoritmo voraz para dispensar el dinero solicitado.

La relación entre las clases es jerárquica: la clase **Main** crea un objeto de la clase **Menu**, y la clase **Menu** utiliza un objeto de la clase **Cajero** para realizar las operaciones del sistema. Esta organización permite separar las responsabilidades del programa y facilita su comprensión y mantenimiento.

Capítulo 7

Casos de Prueba

Para verificar el correcto funcionamiento del sistema se realizaron diferentes pruebas utilizando distintos tipos de entradas.

Caso de prueba 1: Agregar billetes

Entrada:

El usuario ingresa cantidades positivas de billetes en el menú de mantenimiento.

Resultado esperado:

El sistema registra correctamente las cantidades de billetes en el inventario del cajero.

Figura 1

Representación de Casos de prueba

```
Run    Main
"C:\Program Files\Java\jdk-21\bin\java.exe" '

===== CAJERO AUTOMATICO =====
0. Mantenimiento
1. Retirar dinero
2. Salir
0

--- Mantenimiento ---
1. Agregar billetes
2. Ver inventario
3. Salir
1

Ingrese cantidad de billetes:
₡20000: 1
₡10000: 3
₡5000: 4
₡2000: 20
₡1000: 10
 Billetes agregados correctamente
```

```
--- Mantenimiento ---
1. Agregar billetes
2. Ver inventario
3. Salir
2

--- Inventario del Cajero ---
Cantidad de ₡20000: 1
Cantidad de ₡10000: 3
Cantidad de ₡5000: 4
Cantidad de ₡2000: 20
Cantidad de ₡1000: 10
```

Nota: Imagen recortada para mejorar calidad visual, número de figura referente al número de caso según formato

Caso de prueba 2: Retiro exitoso

Entrada:

El usuario solicita retirar ₡38000.

Resultado esperado:

El sistema muestra la combinación de billetes que suman el monto solicitado y descuenta los billetes del inventario.

Figura 2

Representación de Casos de prueba

```
===== CAJERO AUTOMATICO =====
0. Mantenimiento
1. Retirar dinero
2. Salir
1

Ingrese monto a retirar:
38000

--- Billetes entregados ---
Cantidad de ₡20000: 1
Cantidad de ₡10000: 1
Cantidad de ₡5000: 1
Cantidad de ₡2000: 1
Cantidad de ₡1000: 1
----------------------------
Total: ₡38000
```

Nota: Imagen recortada para mejorar calidad visual, número de figura referente al número de caso según formato

Caso de prueba 3: Monto no múltiplo de 1000

Entrada:

El usuario intenta retirar ₡38550.

Resultado esperado:

El sistema muestra un mensaje indicando que el monto debe ser múltiplo de ₡1000.

Figura 3

Representación de Caso de prueba

```
===== CAJERO AUTOMATICO =====
0. Mantenimiento
1. Retirar dinero
2. Salir
1

Ingrese monto a retirar:
38550
 El monto debe ser múltiplo de 1000
```

Nota: Imagen recortada para mejorar calidad visual, número de figura referente al número de caso según formato

Caso de prueba 4: Entrada inválida

Entrada:

El usuario introduce texto en lugar de números.

Resultado esperado:

El sistema solicita nuevamente un número válido sin interrumpir la ejecución del programa.

Figura 4

Representación de caso de prueba

```
===== CAJERO AUTOMATICO =====
0. Mantenimiento
1. Retirar dinero
2. Salir
jkfjsldhj
Debe ingresar un número válido
1

Ingrese monto a retirar:
hola
Debe ingresar un número válido
0
El monto debe ser mayor a 0
```

Nota: Imagen recortada para mejorar calidad visual, número de figura referente al número de caso según formato

Caso de prueba 5: Fondos insuficientes

Entrada:

El usuario solicita una cantidad mayor a la disponible en el cajero automático.

Resultado esperado:

El sistema muestra el mensaje indicando que es imposible dispensar la cantidad solicitada.

Figura 5

Representación de caso de prueba

```
===== CAJERO AUTOMATICO =====
0. Mantenimiento
1. Retirar dinero
2. Salir
1

Ingrese monto a retirar:
1000000
 Imposible dispensar la cantidad solicitada
```

Nota: Imagen recortada para mejorar calidad visual, número de figura referente al número de caso según formato

Capítulo 8

Recomendaciones

Como trabajo futuro, el sistema podría mejorarse incorporando nuevas funcionalidades que amplíen sus capacidades y lo acerquen más al funcionamiento de un cajero automático real.

Entre las mejoras posibles se encuentra la implementación de una interfaz gráfica de usuario que facilite la interacción con el sistema, así como la integración con una base de datos para almacenar el inventario de billetes de manera persistente.

También se podría incorporar un sistema de autenticación de usuarios que permita simular el uso de tarjetas bancarias y códigos de seguridad, lo que aumentaría el nivel de realismo del sistema.

Finalmente, se podrían explorar otros tipos de algoritmos para la dispensación de billetes que permitan optimizar el uso del inventario del cajero automático.

Lista de Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Algorithm Design and Applications*. Wiley.

Git Hub

Enlace al repositorio de Git Hub